

Portare Agenti in produzione con Amazon Bedrock AgentCore

Dal prototipo locale al deploy enterprise: un viaggio attraverso la suite di servizi che semplifica il deploy di agenti AI a scala di produzione.



Riccardo Marostica

Cloud Engineer & Developer

riccardo.marostica@corley.it

corley.it





Riconosciuta come uno dei **top partner AWS in Italia**, Corley Cloud aiuta le aziende a realizzare progetti in cloud innovativi.

Grazie ad un know-how fortemente specializzato nel cloud, a numerose certificazioni ottenute, e ad una partnership pluriennale con AWS, offriamo servizi professionali nei contesti **cloud native, digital cloud transformation, IoT, serverless e machine learning**.



Agenda

01

Il Pain Point

Perché portare agenti in produzione è difficile

02

Cos'è AgentCore

Una piattaforma agentica modulare e serverless

03

I componenti

Runtime, Memory, Gateway, Identity e altri

04

Demo & codice

Un agente di customer support, dal locale al deploy

05

Considerazioni di produzione

Pricing, sicurezza, compliance

06

Takeaway & Q&A

Cosa portarsi a casa

Cos'è l'Agentic AI?



Ragiona

Pianifica strategie e scompone task complessi



Usa tool

Chiama API, funzioni e servizi esterni



Ricorda

Mantiene contesto tra le interazioni

AGENTIC AI



Esegue codice

Scrive ed esegue codice in sandbox sicuri



Naviga

Interagisce con il web e le applicazioni



Collabora

Comunica con altri agenti e con gli utenti

Demo-to-production gap



Computing & Scaling

Sessioni concorrenti, isolamento tra utenti, cold start, gestione dei carichi



Memoria persistente

Gli LLM sono stateless: ogni conversazione riparte da zero senza un layer di memoria



Integrazioni

Connettere API, Lambda, sistemi enterprise a tool MCP-compatibili senza riscrivere tutto



Identity & Auth

Inbound: chi può chiamare l'agente.
Outbound: come l'agente accede a Slack, GitHub, ecc.



Osservabilità

Tracciare reasoning steps, tool calls e model interactions per debug e audit



Security review

Senza isolamento, audit trail e identity, l'agente non passa la review e resta in PoC

Conseguenze concrete



PoC eterni

L'agente resta in demo, non passa la security review



Time-to-market lungo

Mesi di setup prima che il valore di business si manifesti



Costi operativi alti

Team dedicato a mantenere infrastruttura custom



Rischio tecnologico

Lock-in su stack proprietari difficili da evolvere

Amazon Bedrock AgentCore



Framework-agnostic

LangGraph · CrewAI · Strands · LlamaIndex



Model-agnostic

Bedrock, Anthropic, OpenAI, Gemini, ...



Modulare

Adotti i servizi che ti servono, quando ti servono



Memory

Persistenza & strategie di estrazione



Gateway

API, Lambda, OpenAPI → tool MCP



Identity

Inbound + outbound auth (OAuth, OIDC)



RUNTIME

Serverless · microVM isolate · 8h workload



Browser Tool

Browser cloud sicuro per agenti web



Code Interpreter

Esecuzione codice in sandbox



Observability

CloudWatch + OpenTelemetry tracing

Runtime: il cuore della piattaforma

Ambiente serverless dove gira il codice del tuo agente. Nessuna infrastruttura da gestire.

microVM

isolate per ogni sessione

8h

workload di lunga durata

100MB

payload (testo, audio, video)

WebSocket

streaming bidirezionale



Isolamento per sessione

Ogni utente in microVM dedicata: zero contaminazione tra sessioni



Multi-protocollo

HTTP, WebSocket, MCP, Agent-to-Agent (A2A)



Auto-scaling

Scala automaticamente con il carico, paghi solo per l'uso effettivo



SDK unico

Un solo SDK Python per Runtime + Memory + Gateway

Memory: contesto persistente per gli agenti

Gli LLM sono stateless. Memory dà al tuo agente la capacità di ricordare.

MEMORIA A BREVE TERMINE

Contesto della conversazione corrente

Mantiene lo stato all'interno di una sessione utente, gestita automaticamente dal Runtime.

MEMORIA A LUNGO TERMINE

Conoscenza che persiste tra le sessioni

Estrazione automatica di fatti, preferenze e summary in base alla strategia configurata.

Strategie di estrazione configurabili

SEMANTIC

Estrae fatti rilevanti per ricerca semantica

SUMMARY

Crea riassunti delle conversazioni passate

USER_PREFERENCES

Tiene traccia delle preferenze dell'utente

CUSTOM

Definisci la tua strategia di estrazione

Gateway: trasforma le tue API in tool per agenti

Le tue API esistenti diventano tool MCP-compatibili. Senza riscrivere codice.

AWS Lambda

Funzioni esistenti

OpenAPI Spec

REST API documentate

MCP Server

Server MCP esistenti



AgentCore

GATEWAY

Auto-discovery + auth



IL TUO AGENTE

Tool MCP

disponibili nativamente



Niente più boilerplate per esporre tool: il Gateway gestisce semantic search, autenticazione e protocollo MCP per te.

Identity, Browser, Code Interpreter, Observability

Gli altri servizi che completano il quadro production-ready



Identity

Inbound + Outbound Auth

Integrazione con Cognito, Okta, Entra ID. Gestisce sia chi può chiamare l'agente, sia come l'agente accede a Slack, GitHub, Zoom.



Browser Tool

Browser cloud sicuro

Permette all'agente di navigare il web in un sandbox isolato. Utile per agenti che fanno scraping, ricerche o automazioni web.



Code Interpreter

Esecuzione codice in sandbox

L'agente può scrivere ed eseguire codice in modo sicuro per task come analisi dati, calcoli o trasformazioni.



Observability

CloudWatch + OpenTelemetry

Traccia ogni reasoning step, tool invocation e model interaction. Dashboard, metriche e debug a livello di produzione.

Dalla teoria al codice: costruiamo un agente di customer support

Customer support Agent

Costruiamo un agente di customer support che sia capace di:

- 01  Riceve domande dagli utenti
- 02  Ricorda interazioni precedenti
- 03  Chiama un'API interna come tool
- 04  È autenticato e sicuro

RUNTIME

MEMORY

GATEWAY

IDENTITY

L'agente "naked" in locale

Strands Agents in 20 righe. Funziona, ma è un giocattolo.

```
from strands import Agent, tool
from strands.models import BedrockModel

@tool
def get_order_status(order_id: str) -> dict:
    fake_db = {
        "4521": {"status": "in_transit", "carrier": "DHL", "eta": "domani"},
        "4522": {"status": "delivered", "carrier": "BRT", "eta": "consegnato"},
        "4523": {"status": "processing", "carrier": "GLS", "eta": "3-4 giorni"},
    }
    if order_id not in fake_db:
        return {"order_id": order_id, "error": "ordine non trovato"}
    return {"order_id": order_id, **fake_db[order_id]}

model = BedrockModel(
    model_id="us.anthropic.claude-sonnet-4-20250514-v1:0",
    region_name="us-west-2",
)

SYSTEM_PROMPT = """Sei un assistente di customer support per un e-commerce.
Aiuti i clienti a controllare lo stato degli ordini.
Quando un cliente menziona un numero d'ordine, usa il tool get_order_status
per recuperare i dettagli aggiornati. Rispondi in modo cortese e conciso."""

agent = Agent(
    model=model,
    system_prompt=SYSTEM_PROMPT,
    tools=[get_order_status],
)
```

Cosa manca per la produzione?

- ⚠ Scaling delle sessioni
- ⚠ Memoria persistente
- ⚠ Tool & integrazioni
- ⚠ Autenticazione
- ⚠ Observability
- ⚠ Deploy infrastruttura

Deploy su AgentCore Runtime

Aggiungiamo il decoratore @app.entrypoint. Poche righe, e l'agente è production-ready.

```
app = BedrockAgentCoreApp()

model = BedrockModel(
    model_id="us.anthropic.claude-sonnet-4-20250514-v1:0",
    region_name="us-west-2",
)

SYSTEM_PROMPT = """Sei un assistente di customer support per un e-commerce.
Aiuti i clienti a controllare lo stato degli ordini.
Quando un cliente menziona un numero d'ordine, usa il tool get_order_status
per recuperare i dettagli. Rispondi in modo cortese e conciso."""

agent = Agent(
    model=model,
    system_prompt=SYSTEM_PROMPT,
    tools=[get_order_status],
)

@app.entrypoint
def invoke(payload: dict) -> dict:
    user_message = payload.get("prompt", "")
    if not user_message:
        return {"error": "campo 'prompt' mancante nel payload"}

    response = agent(user_message)
    return {"response": str(response)}
```

Deploy in 2 comandi

```
$ agentcore configure
```

```
$ agentcore launch
```

Interazione con l'agente

```
$ agentcore invoke '{"prompt":
"qual è lo stato del mio ordine
4521?"}'
```

Deploy su AgentCore Runtime

Aggiungiamo il decoratore @app.entrypoint. Poche righe, e l'agente è production-ready.

```
app = BedrockAgentCoreApp()

model = BedrockModel(
    model_id="us.anthropic.claude-sonnet-4-20250514-v1:0",
    region_name="us-west-2",
)

SYSTEM_PROMPT = """Sei un assistente di customer support per un e-commerce.
Aiuti i clienti a controllare lo stato degli ordini.
Quando un cliente menziona un numero d'ordine, usa il tool get_order_status
per recuperare i dettagli. Rispondi in modo cortese e conciso."""

agent = Agent(
    model=model,
    system_prompt=SYSTEM_PROMPT,
    tools=[get_order_status],
)

@app.entrypoint
def invoke(payload: dict) -> dict:
    user_message = payload.get("prompt", "")
    if not user_message:
        return {"error": "campo 'prompt' mancante nel payload"}

    response = agent(user_message)
    return {"response": str(response)}
```

Cosa abbiamo ottenuto?

- ✓ Scaling delle sessioni
- ✓ Tool & integrazioni
- ✓ Autenticazione
- ✓ Observability
- ✓ Deploy infrastruttura

Aggiungere Memory e Gateway

L'agente ora ricorda le sessioni passate e chiama una Lambda esposta come tool MCP.

```
# -----  
# Setup app + modello (creati una sola volta, non per ogni request)  
# -----  
app = BedrockAgentCoreApp()  
  
model = BedrockModel(  
    model_id="us.anthropic.claude-sonnet-4-20250514-v1:0",  
    region_name=REGION,  
)  
  
# -----  
# MCP Client verso il Gateway: scopre i tool disponibili e li espone all'agente.  
# Il Gateway gestisce auth, semantic search dei tool e routing alla Lambda.  
# -----  
def make_mcp_client() -> MCPClient:  
    """Crea un MCPClient autenticato verso il Gateway."""  
    headers = {"Authorization": f"Bearer {GATEWAY_TOKEN}"}  
    return MCPClient(  
        lambda: streamablehttp_client(GATEWAY_URL, headers=headers)  
    )
```

Aggiungere Memory e Gateway

L'agente ora ricorda le sessioni passate e chiama una Lambda esposta come tool MCP.

```
@app.entrypoint
def invoke(payload: dict, context: RequestContext) -> dict:
    user_message = payload.get("prompt", "")
    user_id = payload.get("user_id", "anonymous")
    session_id = context.session_id # auto-generato dal Runtime se non passato

    if not user_message:
        return {"error": "campo 'prompt' mancante"}

    # -----
    # Memory: configuriamo cosa recuperare e da dove.
    # I namespace devono matchare quelli definiti in setup_memory.py.
    # top_k = quante memorie recuperare; relevance_score = soglia minima.
    # -----
    memory_config = AgentCoreMemoryConfig(
        memory_id=MEMORY_ID,
        session_id=session_id,
        actor_id=user_id,
        retrieval_config={
            f"/summaries/{user_id}/{session_id}": RetrievalConfig(
                top_k=3, relevance_score=0.3
            ),
            f"/preferences/{user_id}": RetrievalConfig(
                top_k=5, relevance_score=0.3
            ),
        },
    )

    # Il SessionManager si occupa di:
    # - salvare ogni turno come evento Memory (short-term)
    # - recuperare le memorie rilevanti (long-term) e iniettarle nel contesto
    session_manager = AgentCoreMemorySessionManager(
        agentcore_memory_config=memory_config,
        region_name=REGION,
    )
```

```
# -----
# Gateway: apriamo la connessione MCP, scopriamo i tool, costruiamo
# l'agente Strands. Il context manager garantisce la chiusura pulita
# della connessione MCP.
# -----
with make_mcp_client() as mcp_client:
    gateway_tools = mcp_client.list_tools_sync()

    agent = Agent(
        model=model,
        system_prompt=SYSTEM_PROMPT,
        tools=gateway_tools,
        session_manager=session_manager,
    )

    response = agent(user_message)

return {
    "response": str(response),
    "session_id": session_id,
    "user_id": user_id,
}
```

Considerazioni di produzione



Pricing pay-per-use

Nessun commitment iniziale, paghi solo i servizi che usi. Modulare per servizio.



VPC connectivity

Disponibile su tutti i servizi: Runtime, Memory, Gateway, Browser, Code Interpreter.



Compliance enterprise

SOC, ISO 27001/27017/27018, PCI, HIPAA-eligible, ENS High e altri standard regolati.



Coesistenza con Bedrock Agents

Chi usa Bedrock Agents "classici" oggi può continuare. AgentCore è additivo, non sostitutivo.



Built-in tools

Code Interpreter e Browser Tool inclusi. Sandbox per test integrato.



Adozione incrementale

Inizia con Runtime. Aggiungi Memory, poi Gateway, poi Identity. Niente big bang.

Cosa portarsi a casa

01

Il vero collo di bottiglia è il deploy, non lo sviluppo

Costruire un agente è facile. Portarlo in produzione con scaling, memoria, integrazioni e auth è il lavoro vero.

02

AgentCore è modulare e si adotta incrementalmente

Non è un "all-in": parti da Runtime, aggiungi Memory quando ti serve, poi Gateway, poi Identity.

03

Framework e model agnostic

LangGraph, CrewAI, Strands, LlamaIndex e qualsiasi LLM (anche fuori da Bedrock). Lo stack lo scegli tu.

**Thanks
for listening!**

